

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO3:Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5
Duration: 1 Hour

Total Marks:25

CASE STUDY

Code of Conduct:

*I **GIRIJA B/ 192311044** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIRIJA B

1. Designing a Grammar for Arithmetic Expressions

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as $a+b*c$.

Case Study Question:

Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

Context-Free Grammar (CFG)

To correctly enforce **operator precedence** and **associativity**, the grammar can be designed using separate non-terminals for expressions, terms, and factors.

Optimized CFG:

$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

Where:

- E = Expression
- T = Term
- F = Factor
- id = identifier or number

Operator Precedence

The grammar gives multiplication and division higher precedence than addition and subtraction.

$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

For example:

$a + b * c$

The grammar first recognizes:

$b * c$

as a **Term**, and then combines it with a using $+$.

Therefore, the expression is interpreted as:

$a + (b * c)$

rather than:

$(a + b) * c$

Associativity

The recursive structure also establishes **left associativity**.

For example:

$a - b - c$

is interpreted as:

$(a - b) - c$

Similarly:

$a / b / c$

is interpreted as:

$(a / b) / c$

Role of Parentheses

Parentheses are handled by:

$$F \rightarrow (E) \mid \text{id}$$

This allows expressions inside parentheses to be evaluated first.

For example:

$(a + b) * c$

is parsed as:

$(a + b) * c$

because the entire $(a + b)$ is treated as a single factor.

How the Grammar Removes Ambiguity

The original ambiguous grammar may treat $+$ and $*$ at the same level, allowing multiple parse trees for:

$a + b * c$

The optimized grammar separates operators into different levels:

Non-terminal	Operators	Precedence
E	$+, -$	Lower
T	$*, /$	Higher
F	Parentheses, operands	Highest

Thus:

Parentheses $\rightarrow$ Multiplication/Division $\rightarrow$ Addition/Subtraction

This hierarchy ensures that each valid expression has a unique intended interpretation.

Efficient Parsing

The grammar is structured into clear precedence levels, making it suitable for parser implementation.

However, the grammar as written is **left-recursive**, so for a recursive-descent parser it can be transformed into:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid - T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid / F T' \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

This version removes **left recursion** and is suitable for efficient **top-down/recursive-descent parsing**.

Conclusion

The optimized CFG removes ambiguity by assigning different grammar levels to different operators. Multiplication and division have higher precedence than addition and subtraction, while the recursive structure provides left associativity. Parentheses can override the normal precedence rules. The left-recursion-free version can be efficiently implemented using recursive-descent parsing.

2. Mini Programming Language Parser Development

A startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:

Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

1. Initial CFG

A basic grammar can be defined as:

Program $\rightarrow$ StatementList

$\text{StatementList} \rightarrow \text{StatementList Statement} \mid \text{Statement}$

$\text{Statement} \rightarrow \text{Declaration}$
 $\mid \text{IfStatement}$
 $\mid \text{WhileStatement}$
 $\mid \text{Assignment}$

$\text{Declaration} \rightarrow \text{type id ;}$

$\text{Assignment} \rightarrow \text{id = Expression ;}$

$\text{IfStatement} \rightarrow \text{if (Expression) \{ StatementList \}}$

$\text{WhileStatement} \rightarrow \text{while (Expression) \{ StatementList \}}$

$\text{Expression} \rightarrow \text{Expression + Term}$
 $\mid \text{Expression - Term}$
 $\mid \text{Term}$

$\text{Term} \rightarrow \text{Term * Factor}$
 $\mid \text{Term / Factor}$
 $\mid \text{Factor}$

$\text{Factor} \rightarrow \text{(Expression)}$
 $\mid \text{id}$
 $\mid \text{number}$

$\text{type} \rightarrow \text{int} \mid \text{float} \mid \text{bool}$

2. Problems with the Initial Grammar

The grammar contains **left recursion** in productions such as:

$\text{StatementList} \rightarrow \text{StatementList Statement}$

$\text{Expression} \rightarrow \text{Expression + Term}$

$\text{Expression} \rightarrow \text{Expression - Term}$

$\text{Term} \rightarrow \text{Term * Factor}$

$\text{Term} \rightarrow \text{Term / Factor}$

For example:

$\text{Expression} \rightarrow \text{Expression + Term}$

starts with Expression on both sides. A recursive-descent parser may repeatedly call itself without consuming input, causing inefficient parsing or even infinite recursion.

The grammar can also create parsing conflicts because several productions may compete when the parser encounters the same input symbol.

3. Optimized CFG

Left recursion can be eliminated by introducing auxiliary non-terminals.

$\text{Program} \rightarrow \text{StatementList}$

$\text{StatementList} \rightarrow \text{Statement StatementList}'$

$\text{StatementList}' \rightarrow \text{Statement StatementList}' \mid \epsilon$

$\text{Statement} \rightarrow \text{Declaration}$
 $\quad \mid \text{Assignment}$
 $\quad \mid \text{IfStatement}$
 $\quad \mid \text{WhileStatement}$

$\text{Declaration} \rightarrow \text{Type id} ;$

$\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{bool}$

$\text{Assignment} \rightarrow \text{id} = \text{Expression} ;$

$\text{IfStatement} \rightarrow \text{if} (\text{Expression}) \{ \text{StatementList} \}$

$\text{WhileStatement} \rightarrow \text{while} (\text{Expression}) \{ \text{StatementList} \}$

$\text{Expression} \rightarrow \text{Term Expression}'$

$\text{Expression}' \rightarrow + \text{Term Expression}'$
 $\quad \mid - \text{Term Expression}'$
 $\quad \mid \epsilon$

$\text{Term} \rightarrow \text{Factor Term}'$

$\text{Term}' \rightarrow * \text{Factor Term}'$
 $\quad \mid / \text{Factor Term}'$
 $\quad \mid \epsilon$

$\text{Factor} \rightarrow (\text{Expression})$
 $\quad \mid \text{id}$
 $\quad \mid \text{number}$

Here, ϵ represents the empty string.

4. How Left Recursion Is Removed

Consider the original:

$\text{Expression} \rightarrow \text{Expression} + \text{Term} \mid \text{Term}$

It is transformed into:

$\text{Expression} \rightarrow \text{Term Expression}'$

$\text{Expression}' \rightarrow + \text{Term Expression}' \mid \epsilon$

Similarly:

$\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Factor}$

becomes:

$\text{Term} \rightarrow \text{Factor Term}'$

$\text{Term}' \rightarrow * \text{Factor Term}' \mid \epsilon$

This allows the parser to consume the input before making another recursive call.

5. Example

Consider the IoT scripting statement:

```
int temperature;  
temperature = temperature + 5;  
  
if (temperature > 30) {  
    temperature = temperature - 2;  
}
```

```
while (temperature < 40) {  
    temperature = temperature + 1;  
}
```

The parser identifies:

Program



StatementList



Declaration



Assignment



IfStatement



WhileStatement

Nested statements are handled through:

```
if ( Expression ) { StatementList }
```

and

```
while ( Expression ) { StatementList }
```

Thus, multiple declarations, assignments, conditions, and loops can be parsed systematically.

6. Advantages of the Optimized Grammar

Feature	Initial CFG	Optimized CFG
Left recursion	Present	Removed
Recursive-descent parsing	Difficult	Suitable
Parsing conflicts	More likely	Reduced
Expression handling	Ambiguous/complex	Structured
Operator precedence	Less clearly defined	Clearly defined
Parser efficiency	Lower	Higher
Error detection	More difficult	Easier

7. Improvement in Parser Efficiency

The optimized grammar improves efficiency because:

1. **Left recursion is eliminated**, preventing infinite recursive calls.
2. **Expression levels are separated**, with Expression, Term, and Factor handling different operators.
3. **Statement alternatives are clearly separated**, reducing parser decision conflicts.
4. **ε-productions** allow optional continuation of statements and expressions.
5. The grammar is suitable for **recursive-descent and LL-style parsing**.

6. The parser can consume tokens progressively instead of repeatedly expanding the same non-terminal.

Conclusion

The optimized CFG provides a clean syntactic structure for the mini IoT scripting language. By eliminating left recursion and separating expressions into different precedence levels, the grammar reduces syntax conflicts and makes parsing faster and more reliable. It is particularly suitable for implementing an **LL(1)-style or recursive-descent parser** for the scripting language.

3. Grammar Optimization for SQL Query Validation

A database company is developing a query validation tool for simplified SQL commands such as **SELECT, FROM, and WHERE**. The original grammar contains redundant productions that increase parsing time.

Case Study Question:

Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

1. Initial CFG

A basic grammar for simplified SQL syntax is:

Query → SelectStmt

SelectStmt → SELECT ColumnList FROM Table
| SELECT ColumnList FROM Table WHERE Condition

ColumnList → Column
| Column , ColumnList

Column → id
| *

Table → id

Condition → id Operator Value

Operator → =
| <
| >
| <=
| >=
| <>

Value → id
| number
| string

For example:

SELECT name, age FROM student WHERE age > 18

can be generated as:

Query

↓
 SelectStmt
 ↓
 SELECT ColumnList FROM Table WHERE Condition
 ↓
 SELECT name, age FROM student WHERE age > 18

2. Problems with the Initial Grammar

The original grammar has some unnecessary duplication.

For example:

SelectStmt → SELECT ColumnList FROM Table
 | SELECT ColumnList FROM Table WHERE Condition

Both productions repeat:

SELECT ColumnList FROM Table

This increases the number of grammar rules and makes the parser perform unnecessary decisions.

The WHERE clause is optional, so it can be represented more efficiently using an optional non-terminal.

3. Optimized CFG

The redundant productions can be removed as follows:

Query → SELECT ColumnList FROM Table WhereClause

WhereClause → WHERE Condition
 | ε

ColumnList → Column ColumnList'

ColumnList' → , Column ColumnList'
 | ε

Column → id
 | *

Table → id

Condition → id Operator Value

Operator → =
 | <
 | >
 | <=
 | >=
 | <>

Value → id
 | number
 | string

Here, ε represents the **empty string**, meaning that the WHERE clause is optional.

4. Example Queries

Query without WHERE

SELECT name FROM student

Parsing:

Query

↓

SELECT ColumnList FROM Table WhereClause

↓

SELECT name FROM student ϵ

Query with WHERE

SELECT name, age FROM student WHERE age > 18

Parsing:

Query

↓

SELECT ColumnList FROM Table WhereClause

↓

SELECT name, age FROM student WHERE Condition

↓

SELECT name, age FROM student WHERE age > 18

Query with all columns

SELECT * FROM employee

Here:

Column $\rightarrow$ *

allows all columns to be selected.

5. Removal of Redundant Productions

Before optimization

SelectStmt $\rightarrow$ SELECT ColumnList FROM Table

| SELECT ColumnList FROM Table WHERE Condition

The common portion is repeated.

After optimization

Query $\rightarrow$ SELECT ColumnList FROM Table WhereClause

WhereClause $\rightarrow$ WHERE Condition | ϵ

The common portion appears only once.

This reduces grammar size and simplifies parser decisions.

6. Handling Multiple Columns

The optimized grammar uses:

ColumnList $\rightarrow$ Column ColumnList'

ColumnList' $\rightarrow$, Column ColumnList'

| ϵ

Therefore, the following query is valid:

SELECT id, name, age FROM student

The parser repeatedly recognizes:

Column $\rightarrow$ id

$\rightarrow$ name

$\rightarrow$ age

with commas separating the columns.

7. How Optimization Improves Parsing

Feature	Original Grammar	Optimized Grammar
Redundant rules	More	Fewer
Common productions	Repeated	Shared
Optional WHERE	Duplicate SELECT rules	Single optional rule
Parser decisions	More	Fewer
Grammar complexity	Higher	Lower
Syntax analysis	Slower	Faster
Maintainability	More difficult	Easier

The optimized grammar allows the parser to process the common SQL structure only once:

SELECT → ColumnList → FROM → Table

It then makes a simple decision:

WHERE ?

↓

Yes → parse Condition

No → ϵ

This reduces unnecessary grammar expansion and helps the syntax analyzer make decisions more quickly.

8. Conclusion

The optimized CFG removes redundant productions by factoring out the common SELECT ... FROM ... structure and representing the WHERE clause as optional. It also provides a clean structure for handling multiple columns and conditions.

Thus, the optimized grammar:

- **Reduces the number of productions**
- **Eliminates unnecessary duplication**
- **Reduces parsing conflicts**
- **Makes syntax analysis faster**
- **Simplifies parser implementation**
- **Improves grammar maintainability**

4. Natural Language Processing Chatbot

A chatbot system processes simple English sentences such as:

Subject + Verb + Object

Example: “Students learn TOC.”

The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:

Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

1. Initial CFG

A basic grammar can be defined as:

$S \rightarrow NP VP$

$NP \rightarrow N$

| Det N

| Adj N

VP $\rightarrow$ V

| V NP

N $\rightarrow$ Students | Teachers | Students

V $\rightarrow$ learn | study | teach

Det $\rightarrow$ the | a

Adj $\rightarrow$ good | intelligent

However, this grammar has a problem. Since VP $\rightarrow$ V is allowed, it can accept sentences such as:
Students learn.

If the chatbot requires **Subject + Verb + Object**, this sentence should be rejected.

It may also accept combinations that are grammatically unsuitable for the intended language.

2. Optimized CFG

Since the chatbot specifically requires the structure **Subject + Verb + Object**, the grammar can be simplified:

S $\rightarrow$ NP VP

NP $\rightarrow$ N

| Det N

VP $\rightarrow$ V NP

N $\rightarrow$ Students

| Teachers

| Students

| Children

V $\rightarrow$ learn

| study

| teach

Det $\rightarrow$ the

| a

NP $\rightarrow$ N

A cleaner version without the duplicate NP $\rightarrow$ N is:

S $\rightarrow$ NP VP

NP $\rightarrow$ N

| Det N

VP $\rightarrow$ V NP

N $\rightarrow$ students

| teachers

| children

V → learn
| study
| teach

Det → the
| a

O → NP

The essential structure is:

S → Subject Verb Object

represented as:

S → NP V NP

For a very simple chatbot, this can be further optimized to:

S → NP V NP

NP → N
| Det N

N → students
| teachers
| children
| TOC
| mathematics

V → learn
| study
| teach

Det → the
| a

3. Valid Sentence Example

Consider:

Students learn TOC.

The parse is:

S
├── NP
│ ├── N → Students
├── V → learn
└── NP
 ├── N → TOC

Therefore:

Students → Subject

learn → Verb

TOC → Object

So the sentence is **accepted**.

4. Another Valid Example

Input:

The students study mathematics.

The derivation is:

S

→ NP V NP

→ Det N V N

→ The students study mathematics

Therefore, the sentence follows:

Subject + Verb + Object

and is accepted.

5. Invalid Sentence Examples

Example 1

Students TOC learn.

The grammar expects:

NP → V → NP

but receives:

NP → NP → V

Therefore, it is **rejected**.

Example 2

Learn students TOC.

The first word is a verb, but the grammar requires a noun phrase first:

S → NP V NP

Therefore, it is **rejected**.

Example 3

Students learn.

The grammar requires:

S → NP V NP

but the object NP is missing.

Therefore, it is **rejected**.

Example 4

Students mathematics learn.

The expected order is:

Subject → Verb → Object

but the input follows:

Subject → Object → Verb

Therefore, it is **rejected**.

6. How the Optimized CFG Improves Accuracy

The optimized grammar restricts the chatbot to the required sentence structure:

S → NP V NP

This provides stricter validation.

Input	Result	Reason
Students learn TOC	☑ Accepted	S + V + O
The students study mathematics	☑ Accepted	S + V + O
Teachers teach students	☑ Accepted	S + V + O

Input	Result	Reason
Students learn	✗ Rejected	Missing object
Learn students TOC	✗ Rejected	Incorrect order
Students TOC learn	✗ Rejected	Incorrect order
Students mathematics learn	✗ Rejected	Verb in wrong position

7. Advantages of the Optimized Grammar

1. Strict SVO structure

Ensures every sentence contains a subject, verb, and object.

2. Improved validation accuracy

Grammatically invalid sentence patterns are rejected.

3. Reduced ambiguity

Each sentence has a clearly defined structural pattern.

4. Simpler parsing

The parser only needs to recognize:

$NP \rightarrow V \rightarrow NP$

5. Faster syntax checking

The simplified grammar has fewer production rules and fewer parsing choices.

6. Easy to extend

Additional nouns, verbs, and determiners can be added without changing the basic structure.

Conclusion

The optimized CFG for the chatbot is:

$S \rightarrow NP \ V \ NP$

$NP \rightarrow N$

| Det N

$N \rightarrow \text{students} \mid \text{teachers} \mid \text{children} \mid \text{TOC} \mid \text{mathematics}$

$V \rightarrow \text{learn} \mid \text{study} \mid \text{teach}$

$\text{Det} \rightarrow \text{the} \mid \text{a}$

The grammar enforces the **Subject + Verb + Object** structure and rejects sentences with missing components or incorrect word order. By removing unnecessary productions and restricting the possible sentence patterns, the chatbot achieves **more accurate and efficient language validation**.

5. Parser Design for HTML-like Markup Language

A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:

Develop and optimize a CFG for validating properly nested tags in the markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

1. Initial CFG

A basic grammar can be written as:

$S \rightarrow \text{Element}$

Element $\rightarrow$ `<tag> Content </tag>`

Content $\rightarrow$ Element

- | Text
- | Content Element
- | Content Text

Text $\rightarrow$ text

However, this grammar does not explicitly ensure that the opening and closing tags contain the **same tag name**. Therefore, it may incorrectly accept:

`<page>Hello</body>`

even though the tags do not match.

2. Optimized CFG

For a simplified markup language with a fixed set of tags such as page, title, and body, we can define:

$S \rightarrow$ Element

Element $\rightarrow$ `<page> Content </page>`

- | `<title> Content </title>`
- | `<body> Content </body>`

Content $\rightarrow$ Element Content

- | Text Content
- | ϵ

Text $\rightarrow$ text

Here, ϵ represents the empty string.

This grammar ensures that each opening tag has its corresponding closing tag.

3. Recursive Nesting

The important production is:

Content $\rightarrow$ Element Content | Text Content | ϵ

Since Element can contain Content, and Content can contain another Element, the grammar supports **recursive nesting**.

For example:

```
<page>
  <title>Hello</title>
  <body>Welcome</body>
</page>
```

can be represented as:

Element

```
└─ <page>
   └─ Content
      └─ Element
         └─ <title> Hello </title>
      └─ Element
```

```
    └─ <body> Welcome </body>
</page>
```

The recursive structure allows elements to be nested to multiple levels.

4. Example of Deep Nesting

Consider:

```
<page>
  <body>
    <title>Hello</title>
  </body>
</page>
```

The derivation is approximately:

S

→ Element

→ <page> Content </page>

→ <page> Element Content </page>

→ <page> <body> Content </body> Content </page>

→ <page> <body> Element Content </body> Content </page>

→ <page> <body> <title> Text </title> Content </body> Content </page>

Thus, the grammar can recursively validate nested elements.

5. Invalid Nested Tags

Consider:

```
<page>
  <body>
    <title>Hello</body>
  </title>
</page>
```

This is invalid because:

```
<title>
```

was opened but:

```
</body>
```

was encountered before </title>.

The CFG does not have a production that allows this structure, so the parser rejects it.

Similarly:

```
<page>
  <title>Hello</body>
</page>
```

is rejected because <title> must close with:

```
</title>
```

6. Valid and Invalid Examples

Markup	Result	Reason
<page>Hello</page>	✓ Valid	Matching tags
<page><title>Hello</title></page>	✓ Valid	Correct nesting

Markup	Result	Reason
<code><page><body>Text</body></page></code>	✓ Valid	Correct nesting
<code><page><title>Hello</page></title></code>	✗ Invalid	Incorrect nesting
<code><page>Hello</body></code>	✗ Invalid	Mismatched tags
<code><page><body>Text</page></body></code>	✗ Invalid	Incorrect closing order

7. How the Optimized CFG Improves Parsing

The optimized grammar provides several advantages:

1. Correct tag matching

Separate productions ensure that:

`<page> ... </page>`

`<title> ... </title>`

`<body> ... </body>`

are correctly matched.

2. Recursive nesting

The `Element → ... Content ...` structure allows elements to contain other elements.

3. Fewer unnecessary productions

The content structure is simplified to:

`Content → Element Content`

| Text Content

| ε

This avoids redundant rules.

4. Better error detection

Incorrect closing tags and improperly nested elements cannot be derived from the grammar, allowing the parser to report syntax errors more reliably.

5. Efficient parsing

The parser can process the markup from left to right:

Opening Tag

↓

Content

↓

Nested Element / Text

↓

Closing Tag

This makes the structure predictable and reduces unnecessary parsing decisions.

8. Optimized CFG Summary

The final CFG is:

`S → Element`

`Element → <page> Content </page>`

| <title> Content </title>

| <body> Content </body>

`Content → Element Content`

| Text Content

Text → text

Conclusion

The optimized CFG uses **recursive productions** to represent nested markup elements and ensures that every supported opening tag has the correct closing tag. Invalid structures such as mismatched or incorrectly nested tags cannot be generated by the grammar. Therefore, the optimized grammar improves **parsing reliability, error detection, and syntax validation efficiency** for the simplified HTML-like language.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand